

else if statement

◆ else if Statement in JavaScript (Basic to Advanced)

The `else if` statement is used when you want to check **multiple conditions** one by one.

It works with `if` and optional `else`.

✓ 1. Basic Syntax

js

```
if (condition1) {  
    // runs if condition1 is true  
}  
else if (condition2) {  
    // runs if condition1 is false AND condition2 is true  
}  
else {  
    // runs if all above conditions are false  
}
```

✓ 2. How It Works (Step by Step)

JavaScript checks conditions **from top to bottom**:

 Copy code

1. Check `if`
 2. If false → check first `else if`
 3. If false → check next `else if`
 4. If all false → run `else`
 5. Once one condition is true → rest are skipped
-

✓ 3. Simple Example

js

```
let marks = 75;

if (marks >= 90) {
    console.log("Grade A");
}
else if (marks >= 70) {
    console.log("Grade B");
}
else if (marks >= 50) {
    console.log("Grade C");
}
else {
    console.log("Fail");
}
```

🔍 Output:

CSS

Grade B

Because:

- 75 >= 90 ✗
- 75 >= 70 ✓ → Stop here

📋 Copy code
📋 Copy code

✓ 4. Multiple else if Blocks

You can use as many else if as needed.

js

```
let time = 14;

if (time < 12) {
    console.log("Good Morning");
}
```

```
}  
else if (time < 17) {  
    console.log("Good Afternoon");  
}  
else if (time < 20) {  
    console.log("Good Evening");  
}  
else {  
    console.log("Good Night");  
}
```

✓ 5. Important Rules

◆ 1. Order Matters

Wrong order:

 Copy code

js

```
let num = 85;  
  
if (num >= 50) {  
    console.log("Pass");  
}  
else if (num >= 80) {  
    console.log("Distinction");  
}
```

Output → "Pass" ❌ (wrong logic)

Correct order:

 Copy code

js

```
if (num >= 80) {  
    console.log("Distinction");  
}  
else if (num >= 50) {
```

```
    console.log("Pass");  
}
```

 Copy code

◆ 2. Only First True Condition Runs

js

```
let x = 20;  
  
if (x > 5) {  
    console.log("A");  
}  
else if (x > 10) {  
    console.log("B");  
}
```

Output → "A"

Even though both are true, only the first one runs.

 Copy code

✓ 6. Using Logical Operators

js

```
let age = 25;  
  
if (age >= 18 && age <= 60) {  
    console.log("Working Age");  
}  
else if (age > 60) {  
    console.log("Senior Citizen");  
}  
else {  
    console.log("Minor");  
}
```

✓ 7. Nested else if

js

```
let username = "admin";
let password = "1234";

if (username === "admin") {
  if (password === "1234") {
    console.log("Login Successful");
  } else {
    console.log("Wrong Password");
  }
} else {
  console.log("Invalid User");
}
```

✓ 8. Real-World Example (Temperature)

js

```
let temp = 30;

if (temp > 40) {
  console.log("Very Hot");
}
else if (temp > 30) {
  console.log("Hot");
}
else if (temp > 20) {
  console.log("Normal");
}
else {
  console.log("Cold");
}
```

✓ 9. Difference Between `if` and `else if`

✗ Multiple `if` (all checked)

[Copy code](#)

js

```
let num = 10;  
  
if (num > 5) console.log("A");  
if (num > 8) console.log("B");
```

Output:

[Copy code](#)

css

A
B

✓ Using `else if` (only one runs)

[Copy code](#)

js

```
if (num > 5) console.log("A");  
else if (num > 8) console.log("B");
```

Output:

[Copy code](#)

css

A

[Copy code](#)

✓ 10. When to Use `else if`

Use it when:

- You have **multiple related conditions**
- Only **one result** should execute
- Checking **ranges**



Advanced Concept: Short Form Alternative

Instead of many `else if`, sometimes we use:

- ◆ `switch` statement
- ◆ Ternary operator
- ◆ Object mapping

Example (ternary):

js

```
let age = 20;  
let result = age >= 18 ? "Adult" : "Minor";
```

Copy code



Summary

Keyword	Meaning
<code>if</code>	First condition
<code>else if</code>	Additional conditions
<code>else</code>	Default block
Order	Top → Bottom
Stops when	First true condition

If you want, I can now explain:

- ◆ Nested `else if` deeply
- ◆ `switch` VS `else if`
- ◆ Common mistakes in interviews
- ◆ Real interview coding problems

Tell me what level you want



all conditional statement in js

Great! Let's cover **ALL Conditional Statements in JavaScript** from **basic to advanced** 📌

◆ 1. if Statement

Runs code **only** if condition is true.

js

```
let age = 18;

if (age >= 18) {
  console.log("You are an adult");
}
```

◆ 2. if...else

 Copy code

Runs one block if true, another if false.

js

```
let num = 5;

if (num % 2 === 0) {
  console.log("Even");
} else {
  console.log("Odd");
}
```


◆ 3. if...else if...else

Checks **multiple conditions** in order.

js

```
let marks = 72;

if (marks >= 90) {
  console.log("Grade A");
}
else if (marks >= 75) {
  console.log("Grade B");
}
else if (marks >= 50) {
  console.log("Grade C");
}
else {
  console.log("Fail");
}
```

◆ 4. Nested if

if inside another if .

js

```
let user = "admin";
let pass = "1234";

if (user === "admin") {
  if (pass === "1234") {
    console.log("Login Success");
  } else {
    console.log("Wrong Password");
  }
} else {
  console.log("Invalid User");
}
```

◆ 5. switch Statement

Best for **fixed values** (not ranges).

js

```
let day = 3;

switch (day) {
  case 1:
    console.log("Monday");
    break;
  case 2:
    console.log("Tuesday");
    break;
  case 3:
    console.log("Wednesday");
    break;
  default:
    console.log("Invalid day");
}
```

◆ 6. Ternary Operator (? :)

Short form of `if...else` .

js

```
let age = 16;
let result = age >= 18 ? "Adult" : "Minor";
console.log(result);
```

 Copy code

◆ 7. Logical Short-Circuiting (&& / ||)

Used as conditionals in expressions.

js

```
let isLoggedIn = true;

// && runs right side only if left is true
isLoggedIn && console.log("Welcome");

// || runs right side only if left is false
let username = "";
let name = username || "Guest";
console.log(name);
```

◆ 8. ?? (Nullish Coalescing)

[Copy code](#)

Checks only `null` or `undefined`.

js

```
let user = null;
let name = user ?? "Anonymous";
console.log(name);
```

[Copy code](#)

◆ 9. try...catch (Conditional Error Handling)

Executes code safely if error occurs.

js

```
try {
  let x = y + 10; // y not defined
} catch (err) {
  console.log("Error caught:", err.message);
}
```

[Copy code](#)

◆ 10. if with Comparison & Logical Operators

js

```
let age = 25;

if (age >= 18 && age <= 60) {
  console.log("Working Age");
} else if (age > 60) {
  console.log("Senior");
} else {
  console.log("Minor");
}
```

◆ 11. Object-Based Conditional (Advanced Pattern)

[Copy code](#)

Instead of many `else if`:

js

```
let role = "admin";

let access = {
  admin: "Full Access",
  editor: "Edit Access",
  user: "Read Only"
};

console.log(access[role] || "No Access");
```

◆ 12. Function-Based Conditional Logic

js

```
function getDiscount(type) {
  if (type === "student") return 20;
  if (type === "senior") return 30;
  return 0;
}
```

```
console.log(getDiscount("student"));
```

 Copy code

 Summary Table

Type	Use Case
if	Single condition
if...else	True / False
else if	Multiple conditions
nested if	Conditions inside conditions
switch	Fixed values
? :	Short if-else
&& , `	
??	Null/undefined fallback
try...catch	Error-based flow
Object map	Clean multi-case logic

If you want, next I can explain:

-  Conditional Statements **with Real Interview Questions**
-  Flowchart of JS Conditions
-  Tricky edge cases (truthy/falsy, == vs ===)
-  Practice problems with answers 

    ...

else statement all concept in js

Great choice 👍 Let's go deep into the `else` statement in JavaScript — from basic to advanced concepts.

◆ 1. What is `else` ?

`else` runs only when the `if` condition is false.

Basic idea:

- 👉 If something is true → run `if`
 - 👉 Otherwise → run `else`
-

◆ 2. Basic Syntax

js

```
if (condition) {  
    // runs when condition is true  
} else {  
    // runs when condition is false  
}
```

◆ 3. Simple Example

 Copy code

js

```
let age = 16;  
  
if (age >= 18) {  
    console.log("You can vote");  
}
```

```
} else {  
  console.log("You cannot vote");  
}
```

✓ If age $\geq 18 \rightarrow$ if runs

✗ If not $\rightarrow$ else runs

 Copy code

◆ 4. Flow of Execution

js

```
let x = 10;  
  
if (x > 20) {  
  console.log("Big");  
} else {  
  console.log("Small");  
}
```

Steps:

1. Check $x > 20 \rightarrow$ false
2. So else block runs

 Copy code

◆ 5. else with Complex Conditions

js

```
let marks = 40;  
  
if (marks >= 50 && marks <= 100) {  
  console.log("Pass");  
} else {  
  console.log("Fail");  
}
```

◆ 6. else with Truthy & Falsy

 Copy code

In JS, not only `true/false` — values are evaluated as **truthy or falsy**.

Falsy values:

js

```
false, 0, "", null, undefined, NaN
```

Example:

 Copy code

js

```
let username = "";

if (username) {
  console.log("Welcome", username);
} else {
  console.log("Please enter username");
}
```

◆ 7. else Without Braces (Single Line)

 Copy code

js

```
let num = 5;

if (num > 0)
  console.log("Positive");
else
  console.log("Not Positive");
```

⚠️ Avoid this in large programs — use `{}` for safety.

 Copy code

◆ 8. else with Return (Function Use)

js


```
function checkEven(n) {  
  if (n % 2 === 0) {  
    return "Even";  
  } else {  
    return "Odd";  
  }  
}  
  
console.log(checkEven(7));
```

◆ 9. else in Nested Conditions

[Copy code](#)

js

```
let user = "admin";  
let pass = "1234";  
  
if (user === "admin") {  
  if (pass === "1234") {  
    console.log("Login Success");  
  } else {  
    console.log("Wrong Password");  
  }  
} else {  
  console.log("Invalid User");  
}
```

◆ 10. else vs Ternary Operator

else (normal way):

js

```
let result;  
  
if (age >= 18) {  
  result = "Adult";  
}
```

```
} else {  
  result = "Minor";  
}
```

Ternary (short form):

js

```
let result = age >= 18 ? "Adult" : "Minor";
```

 Copy code

◆ 11. Common Mistakes with else

✗ Mistake 1: Missing braces

js

```
if (x > 5)  
  console.log("A");  
  console.log("B"); // Always runs ✗  
else  
  console.log("C");
```

✓ Correct:

js

```
if (x > 5) {  
  console.log("A");  
  console.log("B");  
} else {  
  console.log("C");  
}
```

✗ Mistake 2: Wrong condition logic

 Copy code

js

```
if (age > 18) { ... }  
else { ... }
```

 Copy code

Better:

js

```
if (age >= 18) { ... }  
else { ... }
```

 Copy code

◆ 12. Real-World Example

js

```
let balance = 300;  
  
if (balance >= 500) {  
  console.log("You can withdraw");  
} else {  
  console.log("Insufficient balance");  
}
```

◆ 13. Interview Tip

 Copy code

💡 You cannot use `else` alone.

It must come **after an** `if` **or** `else if`.

✓ Valid:

js

```
if (...) { ... }  
else { ... }
```

✗ Invalid:

 Copy code

js

```
else { ... } // Error
```

 Copy code

 Summary

Feature	Meaning
else	Runs when if is false
Position	After if / else if
Executes	Only one block
Purpose	Default action

If you want, next I can explain:

-  else if in more depth
-  Difference: if vs else vs else if
-  Practice problems with answers
-  Interview trick questions 😎

    ...

nested if else all concept explain i8n js

Perfect 👍 Let’s deeply understand **Nested if...else in JavaScript (Basic → Advanced)** in a clear and structured way.

◆ **1. What is Nested if...else ?**

👉 Nested means “inside another.”

A **nested if...else** is when you put an **if** or **if...else** block **inside another if or else block**.

◆ 2. Basic Structure

js

```
if (condition1) {  
  
    if (condition2) {  
        // runs if condition1 AND condition2 are true  
    } else {  
        // runs if condition1 true but condition2 false  
    }  
  
} else {  
    // runs if condition1 is false  
}
```

◆ 3. Simple Example

js

```
let age = 20;  
let hasID = true;  
  
if (age >= 18) {  
    if (hasID) {  
        console.log("You can enter");  
    } else {  
        console.log("ID required");  
    }  
} else {  
    console.log("You are underage");  
}
```

How it works step-by-step:

1. Check `age >= 18`
2. If true → go inside
3. Then check `hasID`
4. If false → run inner `else`

 Copy code

◆ 4. Why Use Nested `if` ?

We use nested conditions when:

- ✓ One condition depends on another
- ✓ Multi-level decision making
- ✓ Authentication systems
- ✓ Validation systems

◆ 5. Real-World Example (Login System)

js

```
let username = "admin";
let password = "1234";

if (username === "admin") {

    if (password === "1234") {
        console.log("Login Successful");
    } else {
        console.log("Wrong Password");
    }

} else {
    console.log("Invalid Username");
}
```

◆ 6. Multiple Nested Levels

You can nest multiple levels (but avoid too deep nesting).

js

```
let num = 10;

if (num > 0) {

    if (num % 2 === 0) {

        if (num > 5) {
            console.log("Positive, Even, Greater than 5");
        }

    }

}
```

⚠ Too much nesting makes code hard to read.

◆ 7. Flow Diagram (Logic)

Example:

 Copy code

js

```
if (A) {
    if (B) {
        // Code1
    } else {
        // Code2
    }
} else {
    // Code3
}
```

Possible paths:

A	B	Output	 Copy code
true	true	Code1	
true	false	Code2	
false	-	Code3	

◆ 8. Nested if vs Logical Operators

Instead of nested:

js

```
if (age >= 18) {
  if (hasID) {
    console.log("Allowed");
  }
}
```

You can simplify:

js

```
if (age >= 18 && hasID) {
  console.log("Allowed");
}
```

💡 Use logical operators when conditions are simple.

 Copy code

◆ 9. Nested if...else if...else

js

```
let marks = 85;
let attendance = 90;

if (marks >= 50) {
```



```
if (attendance >= 75) {  
  
    if (marks >= 80) {  
        console.log("Distinction");  
    } else {  
        console.log("Pass");  
    }  
  
} else {  
    console.log("Low Attendance");  
}  
  
} else {  
    console.log("Fail");  
}
```

◆ 10. Common Mistakes

✗ 1. Missing Braces

js

```
if (x > 5)  
    if (y > 5)  
        console.log("A");  
else  
    console.log("B"); // Confusing!
```

Always use `{ }` to avoid errors.

 Copy code

✗ 2. Too Much Nesting (Code Pyramid)

js

```
if (A) {  
    if (B) {
```

```
    if (C) {  
        if (D) {  
            // Hard to read  
        }  
    }  
}
```

Better approach:

- Use early returns
- Use logical operators
- Break into functions

 Copy code

◆ 11. Advanced Pattern: Early Return (Best Practice)

Instead of deep nesting:

✖ Bad:

js

```
function checkUser(user) {  
    if (user) {  
        if (user.isActive) {  
            console.log("Access granted");  
        }  
    }  
}
```

✔ Better:

js

```
function checkUser(user) {  
    if (!user) return;  
    if (!user.isActive) return;
```

```
    console.log("Access granted");  
}
```

Cleaner and professional 👍

 Copy code

◆ 12. When to Use Nested if

Use when:

- ✓ Decision depends on previous decision
- ✓ Complex validation
- ✓ Multi-step authentication
- ✓ Menu-based systems

Avoid when:

- ✗ Simple condition (use `&&` or `||`)
- ✗ Too many levels

Final Summary

Concept	Meaning
Nested if	if inside another if
Purpose	Multi-level decision
Risk	Deep nesting = unreadable
Alternative	Logical operators / Early return

If you want next:

 I can give:

- 10 Practice problems (easy → hard)
- Interview tricky nested questions

- Real project example (ATM system)
- Flowchart explanation visually

Tell me what level you want 🚀



10 Practice problems (easy → hard)

Awesome 💪 Let's do **10 Practice Problems on Conditional Statements in JavaScript (Easy → Hard)**

Try them yourself first, then check answers below 🙋

🟢 Easy Level

1 Check Positive or Negative

Write a program to check if a number is positive or negative.

2 Check Even or Odd

Check whether a number is even or odd.

3 Age Group

If age < 13 → Child

If age between 13–19 → Teen

Else → Adult

🟡 Medium Level

4 Grade System

Input marks (0–100)

- 90+ → A
 - 75–89 → B
 - 50–74 → C
 - Below 50 → Fail
-

5 Login Validation

If username is "admin" and password is "1234" → Login Success

Else → Login Failed

6 Largest of Three Numbers

Find the largest among 3 numbers using if...else .

● Hard Level

7 ATM Withdrawal

Conditions:

- Balance must be $\geq$ withdrawal amount
 - Withdrawal amount must be multiple of 100
 - If both true → Allow
Else → Reject
-

8 Electricity Bill

Units:

- First 100 → ₹5/unit
- Next 100 → ₹7/unit
- Above 200 → ₹10/unit

Calculate total bill using conditionals.

9 Leap Year Checker

A year is leap if:

- Divisible by 4
- Not divisible by 100 unless divisible by 400

10 Password Strength Checker

Rules:

- Length ≥ 8
- Must contain at least one number
- Must contain at least one uppercase letter

Print: "Strong" or "Weak"

✓ Solutions (Check After Trying)

✓ 1 Positive or Negative

js

```
let num = -5;

if (num >= 0) {
  console.log("Positive");
} else {
  console.log("Negative");
}
```

✓ 2 Even or Odd

 Copy code

js

```
let num = 7;

if (num % 2 === 0) {
  console.log("Even");
} else {
  console.log("Odd");
}
```

✓ 3 Age Group

[Copy code](#)

js

```
let age = 16;

if (age < 13) {
  console.log("Child");
} else if (age <= 19) {
  console.log("Teen");
} else {
  console.log("Adult");
}
```

✓ 4 Grade System

js

```
let marks = 78;

if (marks >= 90) {
  console.log("A");
} else if (marks >= 75) {
  console.log("B");
} else if (marks >= 50) {
  console.log("C");
} else {
```

```
    console.log("Fail");  
}
```

✓ 5 Login Validation

[Copy code](#)

js

```
let username = "admin";  
let password = "1234";  
  
if (username === "admin" && password === "1234") {  
    console.log("Login Success");  
} else {  
    console.log("Login Failed");  
}
```

✓ 6 Largest of Three

[Copy code](#)

js

```
let a = 10, b = 25, c = 15;  
  
if (a >= b && a >= c) {  
    console.log(a + " is largest");  
} else if (b >= a && b >= c) {  
    console.log(b + " is largest");  
} else {  
    console.log(c + " is largest");  
}
```

✓ 7 ATM Withdrawal

js


```
let balance = 5000;
let withdraw = 1200;

if (withdraw % 100 === 0) {
  if (balance >= withdraw) {
    console.log("Withdrawal Allowed");
  } else {
    console.log("Insufficient Balance");
  }
} else {
  console.log("Enter amount in multiples of 100");
}
```

✓ 8 Electricity Bill

[Copy code](#)

js

```
let units = 250;
let bill = 0;

if (units <= 100) {
  bill = units * 5;
} else if (units <= 200) {
  bill = 100 * 5 + (units - 100) * 7;
} else {
  bill = 100 * 5 + 100 * 7 + (units - 200) * 10;
}

console.log("Bill = ₹" + bill);
```

✓ 9 Leap Year

js

```
let year = 2024;
```

```
if ((year % 4 === 0 && year % 100 !== 0) || year % 400 === 0) {  
    console.log("Leap Year");  
} else {  
    console.log("Not a Leap Year");  
}
```

✓ 10 Password Strength

[Copy code](#)

js

```
let password = "Abc12345";  
  
let hasUpper = false;  
let hasNumber = false;  
  
for (let ch of password) {  
    if (ch >= 'A' && ch <= 'Z') hasUpper = true;  
    if (ch >= '0' && ch <= '9') hasNumber = true;  
}  
  
if (password.length >= 8 && hasUpper && hasNumber) {  
    console.log("Strong Password");  
} else {  
    console.log("Weak Password");  
}
```

If you want next 🗣️

- 👉 Same 10 problems **without solutions** (for exam practice)
- 👉 Flowchart-based problems
- 👉 Interview conditional puzzles
- 👉 Mini project: ATM / Login System / Grading System 😎

[Copy code](#)

📄 👍 💬 ⬆️ ...

hard question with code\

Awesome 🤩 You want **HARD** conditional problems with code in JavaScript.

Here are 5 **Advanced / Tricky Conditional Questions with Full Code + Explanation** 📌

1 Multi-Level Login System (Role + Status)

Problem:

User has:

- role : "admin" | "editor" | "user"
- isActive : true | false

Rules:

- If not active → "Account Disabled"
- If active:
 - admin → "Full Access"
 - editor → "Edit Access"
 - user → "Read Only"

✓ Code

js

```
let role = "editor";
let isActive = true;

if (!isActive) {
  console.log("Account Disabled");
} else {
  if (role === "admin") {
    console.log("Full Access");
  } else if (role === "editor") {
    console.log("Edit Access");
  } else if (role === "user") {
```

```

    console.log("Read Only");
  } else {
    console.log("Unknown Role");
  }
}

```

 Concept Used:

Nested if , logical NOT ! , multi-condition flow

2 Smart Discount System

Problem:

 Copy code

Customer has:

- type : "student" | "regular" | "senior"
- amount (purchase)

Rules:

- student → 20% off
- senior → 30% off
- regular → 10% off only if amount $\geq$ 1000

Code

js

```

let type = "regular";
let amount = 1200;
let discount = 0;

if (type === "student") {
  discount = 0.2;
} else if (type === "senior") {
  discount = 0.3;
} else if (type === "regular" && amount >= 1000) {
  discount = 0.1;
}

```

```
let finalAmount = amount - amount * discount;  
console.log("Payable:", finalAmount);
```

 Concept:

Compound conditions, calculation inside conditionals

 Copy code

3 Online Exam Eligibility

Problem:

Student has:

- attendance (%)
- internalMarks

Rules:

- Attendance must be ≥ 75
- Marks must be ≥ 40
- If both true $\rightarrow$ Eligible
Else $\rightarrow$ Not Eligible

Code

js

```
let attendance = 80;  
let internalMarks = 35;  
  
if (attendance >= 75) {  
  if (internalMarks >= 40) {  
    console.log("Eligible for Exam");  
  } else {  
    console.log("Marks too low");  
  }  
} else {  
  console.log("Low Attendance");  
}
```

 Concept:

Nested if with dependent conditions

Traffic Signal Simulation

 Copy code

Problem:

Signal can be:

- "red" → Stop
- "yellow" → Ready
- "green" → Go

If signal is invalid → Error

Code

js

```
let signal = "yellow";

if (signal === "red") {
  console.log("Stop");
} else if (signal === "yellow") {
  console.log("Ready");
} else if (signal === "green") {
  console.log("Go");
} else {
  console.log("Invalid Signal");
}
```

Concept:

Decision chain with fallback (else)

Loan Approval System (Advanced)

 Copy code

Problem:

Applicant has:

- salary
- creditScore
- hasJob (true/false)

Rules:

- Must have job
 - Salary $\geq 30,000$
 - Credit score ≥ 700
- All 3 must be true $\rightarrow$ Approved

✓ Code

js

```
let salary = 40000;
let creditScore = 720;
let hasJob = true;

if (!hasJob) {
  console.log("Rejected: No Job");
} else if (salary < 30000) {
  console.log("Rejected: Low Salary");
} else if (creditScore < 700) {
  console.log("Rejected: Low Credit Score");
} else {
  console.log("Loan Approved");
}
```

💡 Concept:

Early failure checks + clean else-if chain

🔥 Bonus Hard Interview Trick

[↗ Copy code](#)

js

```
let x = 0;

if (x) {
  console.log("True");
} else {
  console.log("False");
}
```

👉 Output: False

Because `0` is **falsy** in JS 🐙

📄 Copy code

If you want next 👉

- ✓ 10 HARD problems without solutions
- ✓ Tricky MCQs for interviews
- ✓ Real project: ATM / Banking / Login system
- ✓ Nested + Ternary mixed problems

Just tell me 🗨️



nested if else

Got it 👉 Let's explain **Nested if...else in JavaScript** clearly and completely (from basic → advanced).

◆ What is Nested if...else ?

👉 A **nested if...else** means:

An **if** or **else** block **inside another if or else block**.

So you are making **decisions inside decisions**.

◆ Basic Syntax

js

```
if (condition1) {  
    if (condition2) {
```



```
    // runs when condition1 && condition2 are true
  } else {
    // runs when condition1 true, condition2 false
  }
} else {
  // runs when condition1 is false
}
```

◆ Simple Example

js

```
let age = 20;
let hasID = false;

if (age >= 18) {
  if (hasID) {
    console.log("You can enter");
  } else {
    console.log("ID required");
  }
} else {
  console.log("You are underage");
}
```

◆ Step-by-Step Flow

1. Check `age >= 18`
2. If false → "You are underage"
3. If true → check `hasID`
4. If true → "You can enter"
5. If false → "ID required"

 Copy code

◆ Real-World Example: Login System

js

```
let username = "admin";
let password = "1234";

if (username === "admin") {
  if (password === "1234") {
    console.log("Login Successful");
  } else {
    console.log("Wrong Password");
  }
} else {
  console.log("Invalid Username");
}
```

◆ Multiple Levels (Deep Nesting)

js

```
let num = 10;

if (num > 0) {
  if (num % 2 === 0) {
    if (num > 5) {
      console.log("Positive, Even, and > 5");
    }
  }
}
```

⚠ Works, but hard to read when too deep.

◆ Better Way (Avoid Too Much Nesting)

 Copy code

Instead of:

js

```
if (age >= 18) {
  if (hasID) {
```

```
        console.log("Allowed");
    }
}
```

Use:

js

```
if (age >= 18 && hasID) {
    console.log("Allowed");
}
```

Or early return:

 Copy code

js

```
function check(age, hasID) {
    if (age < 18) return "Underage";
    if (!hasID) return "ID Required";
    return "Allowed";
}
```

◆ Common Mistakes

 Copy code

✗ Missing {}

js

```
if (x > 5)
    if (y > 5)
        console.log("A");
else
    console.log("B"); // Confusing!
```

✓ Always use {} .

 Copy code

◆ When to Use Nested if...else

- ✔ When one condition depends on another
- ✔ Login systems, validations, permissions
- ✔ Multi-step logic
- ✗ Avoid when simple `&&` / `||` works

 Summary

Point	Meaning
Nested if	<code>if</code> inside <code>if</code>
Purpose	Multi-level decisions
Risk	Too deep = unreadable
Alternative	Logical operators / early return

If you want next:

- ➡ Nested if...else **flowchart**
- ➡ 10 nested practice problems
- ➡ Interview tricky nested questions
- ➡ Real project example (ATM / Login / Grading)

Just tell me 🕶️



logical all concept

Great 👍 Let's cover **ALL Logical Concepts in JavaScript** clearly from **basic** → **advanced**.

◆ 1. What are Logical Operators?

Logical operators are used to **combine conditions** and return `true` or `false` .

Operator	Name	Meaning
<code>&&</code>	AND	All conditions must be true
<code>,</code>		<code>,</code>
<code>!</code>	NOT	Reverses the result

◆ 2. AND (`&&`) Operator

✔ True only if both sides are true

js

```
let age = 25;
let hasID = true;

if (age >= 18 && hasID) {
  console.log("Allowed");
}
```

A	B	A && B
T	T	T
T	F	F
F	T	F
F	F	F

 Copy code

◆ 3. OR (`||`) Operator

✔ True if any one is true

js

```

let isAdmin = false;
let isEditor = true;

if (isAdmin || isEditor) {
  console.log("Access Granted");
}

```

| A | B | A || B |

|---|---|-----|

| T | T | T |

| T | F | T |

| F | T | T |

| F | F | F |

 Copy code

◆ 4. NOT (!) Operator

✓ Reverses truth value

js

```

let isOnline = false;

if (!isOnline) {
  console.log("User Offline");
}

```

A

!A

T

F

F

T

 Copy code

◆ 5. Logical Operators with Conditions

js

```
let marks = 80;
let attendance = 90;

if (marks >= 50 && attendance >= 75) {
  console.log("Eligible");
}
```

 Copy code

◆ 6. Short-Circuit Behavior (IMPORTANT)

JavaScript stops evaluating when result is decided.

◆ AND (&&)

- If left is `false` → right is NOT checked

js

```
false && console.log("Hi"); // won't run
```

 Copy code

◆ OR (||)

- If left is `true` → right is NOT checked

js

```
true || console.log("Hello"); // won't run
```

 Copy code

◆ 7. Logical Operators Return Values (Not Always Boolean!)

js

```
let result1 = "Hello" && "World"; // "World"
let result2 = "" || "Default"; // "Default"
```

Rules:

 Copy code

- `&&` returns **first falsy** or **last truthy**

- `||` returns **first truthy**

◆ 8. Truthy & Falsy Values

Falsy values in JS:

js

```
false, 0, "", null, undefined, NaN
```

Everything else is truthy.

 Copy code

Example:

js

```
let name = "";

if (!name) {
  console.log("Name is required");
}
```

◆ 9. Nullish Coalescing (??)

 Copy code

Only checks `null` or `undefined` (not `0` or `""`)

js

```
let user = null;
let name = user ?? "Guest"; // "Guest"
```

Difference:

 Copy code

js

```
let x = 0;
x || 10 // 10
x ?? 10 // 0
```


 Copy code

◆ 10. Logical Assignment Operators (ES2021+)

Operator	Meaning
&&=	Assign if left is true

,

??=	Assign if null/undefined
-----	--------------------------

js

```
let a = 0;  
a ||= 10;    // a = 10
```

```
let b = null;  
b ??= 5;     // b = 5
```

 Copy code

◆ 11. Using Logical Ops Instead of if

js

```
let isLoggedIn = true;  
isLoggedIn && console.log("Welcome");
```

```
let username = "";  
let name = username || "Guest";
```

 Copy code

◆ 12. Interview Tricky Example

js

```
console.log(5 && 0 || 3);
```

 Copy code

Evaluate:

- 5 && 0 → 0
- 0 || 3 → 3

✓ Output: 3

Final Summary

Concept	Key Point
&&	All must be true
,	
!	Reverse result
Short-circuit	Stops early
Returns value	Not always boolean
??	Only null/undefined
`&&=	

If you want next:

- 👉 Logical Operators + Conditional statements mix
- 👉 15 MCQs (interview level)
- 👉 Real coding problems using logical operators
- 👉 Tricky output-based questions 😎

    ...

truth falsy all concept

Great topic 👍 Let's cover **Truthy & Falsy in JavaScript — ALL Concepts (Basic → Advanced)** in a simple, deep way.

◆ 1. What are Truthy & Falsy?

In JavaScript, conditions don't only check `true` or `false`.

👉 They check whether a value is **truthy** or **falsy**.

- **Falsy** → treated as `false`
 - **Truthy** → treated as `true`
-

◆ 2. The 7 Falsy Values (IMPORTANT)

Only these values are **falsy** in JS:

```
js  
  
false  
0  
-0  
0n      // BigInt zero  
""      // empty string  
null  
undefined  
NaN
```

⚠ Everything else is **truthy**.

◆ 3. Examples of Truthy

📋 Copy code

```
js  
  
true  
1  
-1  
"hello"
```

```

"0"
"false"
[]
{}
function(){}

```

All are truthy 🕶️

 Copy code

◆ 4. Using Truthy/Falsy in if

js

```

let username = "";

if (username) {
  console.log("Welcome", username);
} else {
  console.log("Please enter username");
}

```

Because "" is falsy → else runs.

 Copy code

◆ 5. Boolean Conversion

JavaScript converts values to boolean automatically in conditions.

You can also convert manually:

js

```

Boolean(0)      // false
Boolean("hello") // true
Boolean([])     // true
Boolean(null)   // false

```

Or using !! trick:

 Copy code

js

```
!!"hi"    // true
!!0       // false
```

 Copy code

◆ 6. Truthy/Falsy with Logical Operators

js

```
let a = "" || "Default"; // "Default"
let b = "Hi" && 10;      // 10
```

Rules:

 Copy code

- || returns first truthy
- && returns first falsy OR last truthy

◆ 7. Common Mistakes

✗ Thinking empty array is falsy:

js

```
if ([]) console.log("Runs"); // YES, runs 🤖
```

✗ Thinking "0" is falsy:

 Copy code

js

```
if ("0") console.log("True"); // YES, runs
```

 Copy code

◆ 8. Truthy/Falsy vs == Comparison

js

```
if (0 == false) // true 🤖
if (0 === false) // false 🕶️
```

 Copy code

Use `===` to avoid confusion.

◆ 9. Real-World Use Cases

✓ Check input:

js

```
if (!email) {  
  console.log("Email required");  
}
```

✓ Default values:

 Copy code

js

```
let name = userInput || "Guest";
```

✓ Toggle flags:

 Copy code

js

```
let isOpen = true;  
isOpen = !isOpen;
```

 Copy code

◆ 10. Edge Cases (Interview Level)

js

```
console.log(Boolean(""));    // false  
console.log(Boolean(" "));   // true (space!)  
console.log(Boolean([]));    // true  
console.log(Boolean({}));    // true  
console.log(Boolean(NaN));    // false
```

 Copy code

Value	Truthy / Falsy
0	✗ Falsy
"0"	✓ Truthy
""	✗ Falsy
" "	✓ Truthy
null	✗ Falsy
undefined	✗ Falsy
[]	✓ Truthy
{}	✓ Truthy

If you want next 📌

- 👉 15 Tricky Truthy/Falsy MCQs
- 👉 Output-based questions
- 👉 Mini project using truthy/falsy
- 👉 Practice problems with answers 😎

📄 👍 💬 ↗️ ...

switch statement

Great 👍 Let's learn `switch` **statement in JavaScript** completely (Basic → Advanced).

◆ 1. What is `switch` ?

`switch` is used to check **multiple possible values** of one variable.

👉 It is mainly used when:

- You compare **one variable**
 - Against **fixed values**
 - Not ranges (like $>$, $<$)
-

◆ 2. Basic Syntax

js

```
switch (expression) {  
  case value1:  
    // code  
    break;  
  
  case value2:  
    // code  
    break;  
  
  default:  
    // runs if no case matches  
}
```

◆ 3. Simple Example

js

```
let day = 2;  
  
switch (day) {  
  case 1:  
    console.log("Monday");  
    break;  
  case 2:  
    console.log("Tuesday");  
    break;  
  case 3:
```



```
    console.log("Wednesday");  
    break;  
default:  
    console.log("Invalid day");  
}
```

👉 Output: Tuesday

◆ 4. How It Works

1. `switch` checks expression
2. Compares with each `case`
3. If matched → runs that block
4. Stops when `break` is found

📋 Copy code

◆ 5. Why `break` is Important

Without `break`, it continues to next case.

Example (no break):

js

```
let x = 1;  
  
switch (x) {  
  case 1:  
    console.log("One");  
  case 2:  
    console.log("Two");  
  case 3:  
    console.log("Three");  
}
```

Output:

sql

One

Two

Three

This is called **fall-through**.

 Copy code

◆ 6. Multiple Cases (Grouping)

js

```
let fruit = "apple";

switch (fruit) {
  case "apple":
  case "mango":
  case "banana":
    console.log("This is a fruit");
    break;
  default:
    console.log("Unknown item");
}
```

✓ All 3 cases share same code.

◆ 7. switch Uses Strict Comparison (===)

 Copy code

Important:

js

```
let num = "1";

switch (num) {
  case 1:
    console.log("Number 1");
    break;
  case "1":
```

```
    console.log("String 1");  
}
```

👉 Output: "String 1"

Because `switch` uses **strict equality** (`===`).

 Copy code

◆ 8. Using `switch` with Strings

js

```
let role = "admin";  
  
switch (role) {  
  case "admin":  
    console.log("Full Access");  
    break;  
  case "user":  
    console.log("Limited Access");  
    break;  
  default:  
    console.log("No Access");  
}
```

◆ 9. Switch with Boolean (Advanced Trick)

You can use `true` to check conditions:

js

```
let marks = 85;  
  
switch (true) {  
  case marks >= 90:  
    console.log("A");  
    break;  
  case marks >= 75:  
    console.log("B");  
}
```

```
    break;
case marks >= 50:
    console.log("C");
    break;
default:
    console.log("Fail");
}
```

👉 Works like if-else .

◆ 10. Real-World Example (Calculator)

js

```
let a = 10;
let b = 5;
let operator = "+";

switch (operator) {
  case "+":
    console.log(a + b);
    break;
  case "-":
    console.log(a - b);
    break;
  case "*":
    console.log(a * b);
    break;
  case "/":
    console.log(a / b);
    break;
  default:
    console.log("Invalid operator");
}
```

◆ 11. When to Use switch vs if-else

Use <code>switch</code>	Use <code>if-else</code>
Fixed values	Ranges (<code>></code> <code><</code>)
Many equal comparisons	Complex logic
Cleaner menu system	Condition-based logic

 Copy code

◆ 12. Common Mistakes

- ✗ Forgetting `break`
- ✗ Using ranges directly (use `switch(true)` instead)
- ✗ Forgetting `default`

 Final Summary

Concept	Meaning
<code>case</code>	Matching value
<code>break</code>	Stops execution
<code>default</code>	Runs if no match
Comparison	Uses <code>===</code>
Fall-through	Happens without <code>break</code>

If you want next 

-  10 Switch Practice Problems
-  Tricky Output-Based Questions
-  Switch vs If-Else Interview Questions
-  Build Mini Calculator Project 

    ...

alert and prompt all concept basic to advanced completet

Perfect 👍 Let's cover `alert()` and `prompt()` in JavaScript — from Basic → Advanced (Complete Guide).

These are part of the **Browser Window API** and used for simple user interaction.

◆ 1. What is `alert()` ?

`alert()` shows a **popup message** to the user.

- ✓ It only has **OK button**
- ✓ It **pauses code execution** until user clicks OK

✓ Syntax

js

```
alert("Hello World!");
```

📋 Copy code

◆ 2. Example of `alert()`

js

```
alert("Welcome to our website!");
```

📋 Copy code

👉 Shows a popup with the message.

◆ 3. What is `prompt()` ?

prompt() is used to:

- ✓ Ask input from the user
- ✓ Returns the user's input as a **string**

✓ Syntax

js

```
let value = prompt("Enter your name:");
```

 Copy code

◆ 4. Example of prompt()

js

```
let name = prompt("What is your name?");  
alert("Hello " + name);
```

 Copy code

◆ 5. Important: prompt() Always Returns String

js

```
let a = prompt("Enter first number");  
let b = prompt("Enter second number");  
  
console.log(a + b); // ✗ string concat
```

Fix it:

js

```
let a = Number(prompt("Enter first number"));  
let b = Number(prompt("Enter second number"));  
  
console.log(a + b); // ✓ number addition
```

 Copy code

◆ 6. Handling Cancel Button

If user clicks **Cancel**, `prompt()` returns `null` .

js

```
let name = prompt("Enter name");

if (name === null) {
    alert("You cancelled");
} else if (name === "") {
    alert("Empty input");
} else {
    alert("Hello " + name);
}
```

◆ 7. Using Default Value in Prompt

js

```
let city = prompt("Enter your city", "Delhi");
```

 Copy code

◆ 8. Using `alert()` + `prompt()` Together

js

```
alert("Welcome!");

let age = prompt("Enter your age");

if (age >= 18) {
    alert("You are eligible");
} else {
    alert("Not eligible");
}
```


◆ 9. Using `confirm()` with Them (Advanced)

`confirm()` gives **OK / Cancel** and returns `true/false` .

[Copy code](#)

js

```
let result = confirm("Are you sure?");
if (result) {
    alert("Confirmed");
} else {
    alert("Cancelled");
}
```

◆ 10. Type Conversion in Prompt (Advanced)

[Copy code](#)

Method	Example
<code>Number()</code>	<code>Number(prompt("Enter number"))</code>
<code>parseInt()</code>	<code>parseInt(prompt("Enter age"))</code>
<code>parseFloat()</code>	<code>parseFloat(prompt("Enter price"))</code>

◆ 11. Validation Using Prompt

js

```
let age = Number(prompt("Enter age"));

if (isNaN(age)) {
    alert("Invalid number");
} else if (age < 0) {
    alert("Age cannot be negative");
} else {
    alert("Your age is " + age);
}
```

◆ 12. Loop + Prompt (Repeated Input)

js

```
let password;

do {
  password = prompt("Enter password:");
} while (password !== "1234");

alert("Login Successful");
```

◆ 13. Limitations (Important)

 Copy code

- ⊘ Not stylish
- ⊘ Blocks UI
- ⊘ Not for production apps
- ⊘ Only for learning / testing

Modern apps use:

- ✓ HTML forms
- ✓ Modal popups
- ✓ Custom UI dialogs

◆ 14. Real-World Mini Example

js

```
alert("ATM System");

let balance = 5000;
let withdraw = Number(prompt("Enter amount to withdraw"));

if (withdraw % 100 !== 0) {
  alert("Enter multiples of 100");
} else if (withdraw > balance) {
  alert("Insufficient Balance");
}
```

```
    } else {  
      balance -= withdraw;  
      alert("Collect Cash\nBalance: " + balance);  
    }  
  }  
}
```

 Summary

Function	Purpose	Returns
alert()	Show message	undefined  Copy code
prompt()	Take input	string or null
confirm()	Yes/No	true/false

- If you want next 📌
- 👉 10 Practice Problems using alert() & prompt()
 - 👉 Mini Project (ATM / Login / Calculator using prompt)
 - 👉 Interview MCQs
 - 👉 Flowchart + logic problems 😎

    ...

10 Practice Problems using alert() & prompt() advanced like

Perfect 😎 Here are 10 Advanced Practice Problems using alert() & prompt() in JavaScript.
These are **logic + validation + conditionals + loops** level — great for interviews & real thinking.

👉 Try to solve them yourself first. (If you want, I'll send full solutions next.)

🔥 1 Smart Login System

Ask user:

- Username
- Password

Rules:

- Username must be "admin"
 - Password must be "1234"
 - If either is wrong → show specific error using `alert()`
 - If correct → "Login Successful"
-

🔥 2 Number Guessing Game

- Generate a fixed number like 7
 - Ask user to guess using `prompt()`
 - If guess is too high → "Too High"
 - Too low → "Too Low"
 - Correct → "You Win!"
-

🔥 3 Age Verification with Validation

Ask user age:

- If input is empty → "Input required"
 - If not a number → "Invalid Age"
 - If < 18 → "Not Allowed"
 - Else → "Welcome"
-

🔥 4 ATM Mini System

Ask:

1. Balance = 5000
2. Ask withdraw amount

Rules:

- Must be number
- Must be multiple of 100
- Must not exceed balance

Show final balance with `alert()` .

5 Password Strength Checker

Ask for password:

Rules:

- Length ≥ 8
- At least one uppercase
- At least one number

Show:

- "Strong Password"
 - Or "Weak Password"
-

6 Electricity Bill Calculator

Ask for units:

Rules:

- First 100 $\rightarrow$ ₹5/unit
- Next 100 $\rightarrow$ ₹7/unit
- Above 200 $\rightarrow$ ₹10/unit

Calculate bill and show in alert.

7 Simple Calculator

Ask:

- First number

- Operator (+ - * /)
- Second number

Perform operation using `alert()` .

Handle:

- Invalid operator
 - Division by 0
-

8 Student Result System

Ask:

- Marks (0–100)
- Attendance (%)

Rules:

- Attendance ≥ 75
- Marks ≥ 40

Show:

- ✓ Pass / Fail
 - ✓ Reason if fail
-

9 Menu-Based System (Loop + Prompt)

Menu:

markdown

1. Check Balance
2. Deposit
3. Withdraw
4. Exit

Use `prompt()` in a loop until user chooses Exit.

 Copy code

10 Multi-Step Registration System

Ask:

- Name
- Email
- Age

Rules:

- No empty input
- Age must be number
- Age ≥ 18

Final alert:

- ✓ Registration Successful
- ✗ Show exact error

Concepts Covered

- ✓ Input validation
- ✓ Type conversion
- ✓ Loops + prompt
- ✓ Nested if
- ✓ Real-life logic
- ✓ Error handling

If you want next 

- ➡ Full **SOLUTIONS** with code
- ➡ Flowchart logic
- ➡ Interview MCQs
- ➡ Mini Project using prompt & alert

Just say: "Send solutions" 



10 Practice Problems using alert() & prompt() with code

Awesome 🙌 Here are **10 Practice Problems using alert() & prompt() with FULL CODE (Advanced Level)**.

You can copy-paste and run each one in your browser console or inside a `<script>` tag.

1 Smart Login System

js

```
let username = prompt("Enter username:");
let password = prompt("Enter password:");

if (username === "admin") {
  if (password === "1234") {
    alert("Login Successful");
  } else {
    alert("Wrong Password");
  }
} else {
  alert("Invalid Username");
}
```

2 Number Guessing Game

js

```
let secret = 7;
let guess = Number(prompt("Guess the number (1-10):"));

if (isNaN(guess)) {
  alert("Invalid input");
} else if (guess > secret) {
```



```
    alert("Too High");  
  } else if (guess < secret) {  
    alert("Too Low");  
  } else {  
    alert("You Win!");  
  }  
}
```

3 Age Verification with Validation

js

```
let age = prompt("Enter your age:");  
  
if (age === null || age === "") {  
  alert("Input required");  
} else {  
  age = Number(age);  
  if (isNaN(age)) {  
    alert("Invalid Age");  
  } else if (age < 18) {  
    alert("Not Allowed");  
  } else {  
    alert("Welcome");  
  }  
}
```

4 ATM Withdrawal System

js

```
let balance = 5000;  
let amount = Number(prompt("Enter amount to withdraw:"));  
  
if (isNaN(amount)) {  
  alert("Invalid amount");  
} else if (amount % 100 !== 0) {  
  alert("Enter amount in multiples of 100");  
}
```

```
} else if (amount > balance) {  
    alert("Insufficient Balance");  
} else {  
    balance -= amount;  
    alert("Withdrawal Successful\nRemaining Balance: " + balance);  
}
```

5 Password Strength Checker

js

```
let password = prompt("Enter password:");  
  
let hasUpper = false;  
let hasNumber = false;  
  
for (let ch of password) {  
    if (ch >= "A" && ch <= "Z") hasUpper = true;  
    if (ch >= "0" && ch <= "9") hasNumber = true;  
}  
  
if (password.length >= 8 && hasUpper && hasNumber) {  
    alert("Strong Password");  
} else {  
    alert("Weak Password");  
}
```

6 Electricity Bill Calculator

js

```
let units = Number(prompt("Enter electricity units:"));  
let bill = 0;  
  
if (isNaN(units) || units < 0) {  
    alert("Invalid units");  
} else {
```

```
if (units <= 100) {  
    bill = units * 5;  
} else if (units <= 200) {  
    bill = 100 * 5 + (units - 100) * 7;  
} else {  
    bill = 100 * 5 + 100 * 7 + (units - 200) * 10;  
}  
alert("Total Bill = ₹" + bill);  
}
```

7 Simple Calculator

js

```
let a = Number(prompt("Enter first number:"));  
let op = prompt("Enter operator (+ - * /):");  
let b = Number(prompt("Enter second number:"));
```

```
let result;
```

```
if (isNaN(a) || isNaN(b)) {  
    alert("Invalid numbers");  
} else {  
    switch (op) {  
        case "+":  
            result = a + b;  
            break;  
        case "-":  
            result = a - b;  
            break;  
        case "*":  
            result = a * b;  
            break;  
        case "/":  
            if (b === 0) {  
                alert("Cannot divide by zero");  
                break;  
            }  
            result = a / b;
```

```
        break;
    default:
        alert("Invalid operator");
    }

    if (result !== undefined) {
        alert("Result: " + result);
    }
}
```

8 Student Result System

js

```
let marks = Number(prompt("Enter marks (0-100):"));
let attendance = Number(prompt("Enter attendance %:"));

if (isNaN(marks) || isNaN(attendance)) {
    alert("Invalid input");
} else if (attendance < 75) {
    alert("Fail: Low Attendance");
} else if (marks < 40) {
    alert("Fail: Low Marks");
} else {
    alert("Pass");
}
```

9 Menu-Based System (Loop + Prompt)

js

```
let balance = 1000;
let choice;

do {
    choice = prompt(
        "Menu:\n1. Check Balance\n2. Deposit\n3. Withdraw\n4. Exit\nEnter choice:"
    );
}
```

```
);

switch (choice) {
  case "1":
    alert("Balance: " + balance);
    break;

  case "2":
    let dep = Number(prompt("Enter deposit amount:"));
    if (!isNaN(dep) && dep > 0) {
      balance += dep;
      alert("Deposited. Balance: " + balance);
    } else {
      alert("Invalid amount");
    }
    break;

  case "3":
    let w = Number(prompt("Enter withdraw amount:"));
    if (w > balance) {
      alert("Insufficient Balance");
    } else if (!isNaN(w) && w > 0) {
      balance -= w;
      alert("Withdrawn. Balance: " + balance);
    } else {
      alert("Invalid amount");
    }
    break;

  case "4":
    alert("Thank You!");
    break;

  default:
    alert("Invalid choice");
}
} while (choice !== "4");
```

10 Multi-Step Registration System

js

```
let name = prompt("Enter your name:");
let email = prompt("Enter your email:");
let age = Number(prompt("Enter your age:"));

if (!name || !email) {
    alert("All fields required");
} else if (isNaN(age)) {
    alert("Invalid age");
} else if (age < 18) {
    alert("Must be 18+");
} else {
    alert("Registration Successful\nName: " + name + "\nEmail: " + email);
}
```

 Copy code

If you want next 🔥

 Copy code

- 👉 Same 10 problems **without solutions**
- 👉 Interview MCQs from `prompt/alert`
- 👉 Mini Project: ATM / Login / Calculator using `prompt`
- 👉 Flowchart logic diagrams

Just tell me 😎

